Seguridad en K8S

CAS TRAINING

BUENAS PRACTICAS SEGURIDAD

- Deshabilitar el acceso root.
- Desactivar la autenticación basada en contraseñas.
- Forzar la autenticación mediante claves SSH.
- Implementar medidas adicionales para asegurar tus sistemas físicos o virtuales.

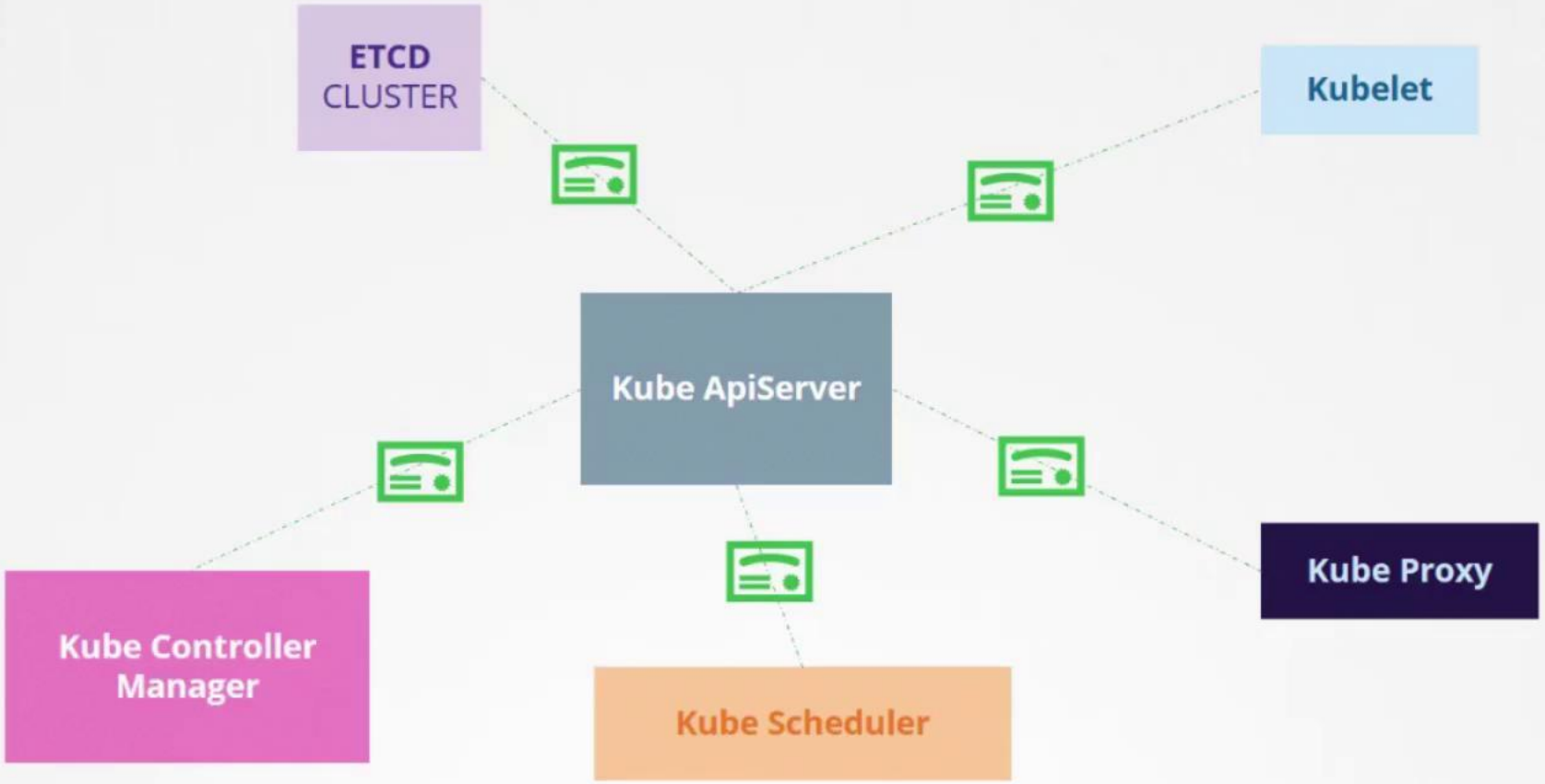
AUTENTICACION

- Kubernetes ofrece diversos mecanismos de autenticación para adaptarse a diferentes necesidades de seguridad:
  - IDs de usuario y contraseñas estáticas
  - Tokens
  - Certificados de cliente
  - Integración con proveedores de autenticación externos (por ejemplo, LDAP)

AUTHORIZATION

- Después de la autenticación, la autorización determina qué acciones puede realizar un usuario o servicio. El mecanismo predeterminado, el Control de Acceso Basado en Roles (RBAC), asocia identidades con permisos específicos. Kubernetes también soporta:
  - Control de Acceso Basado en Atributos (ABAC)
  - Autorización de nodos
  - Autorización basada en webhooks

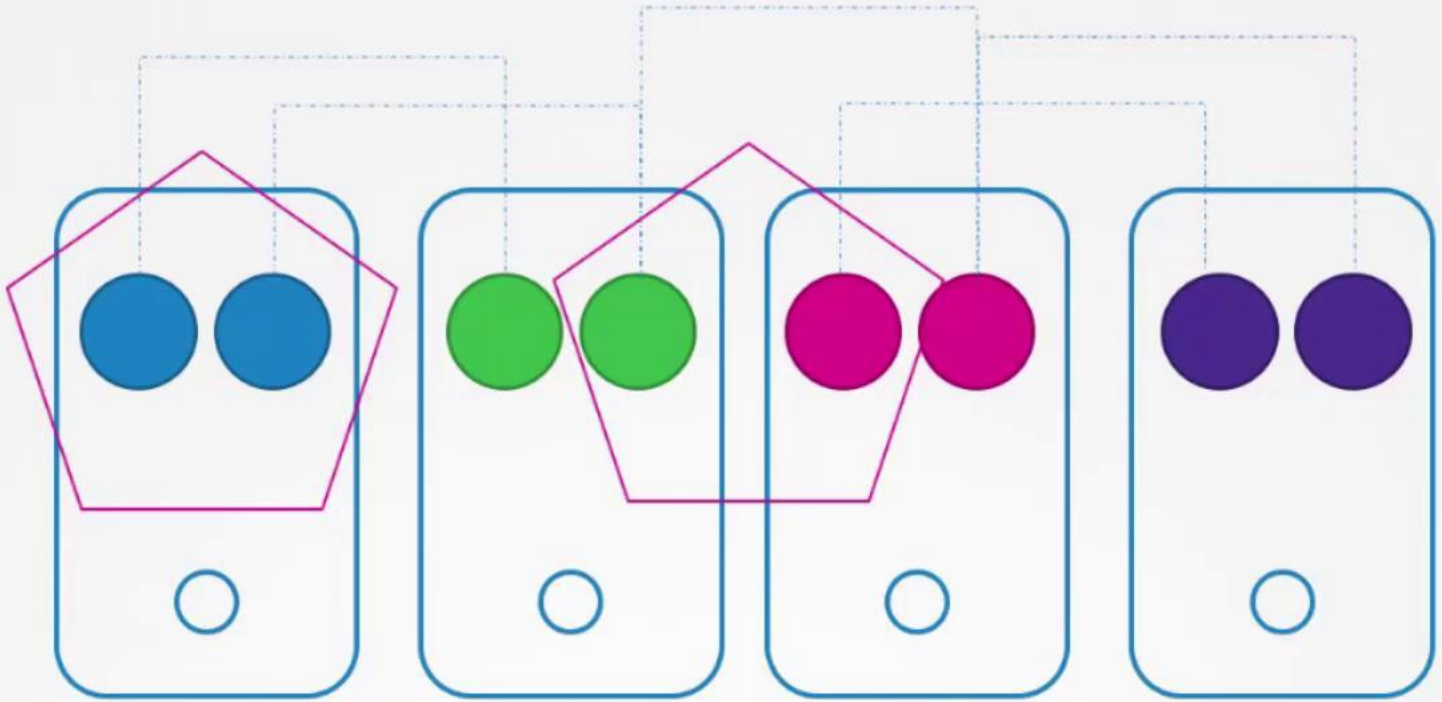
TLS Certificates

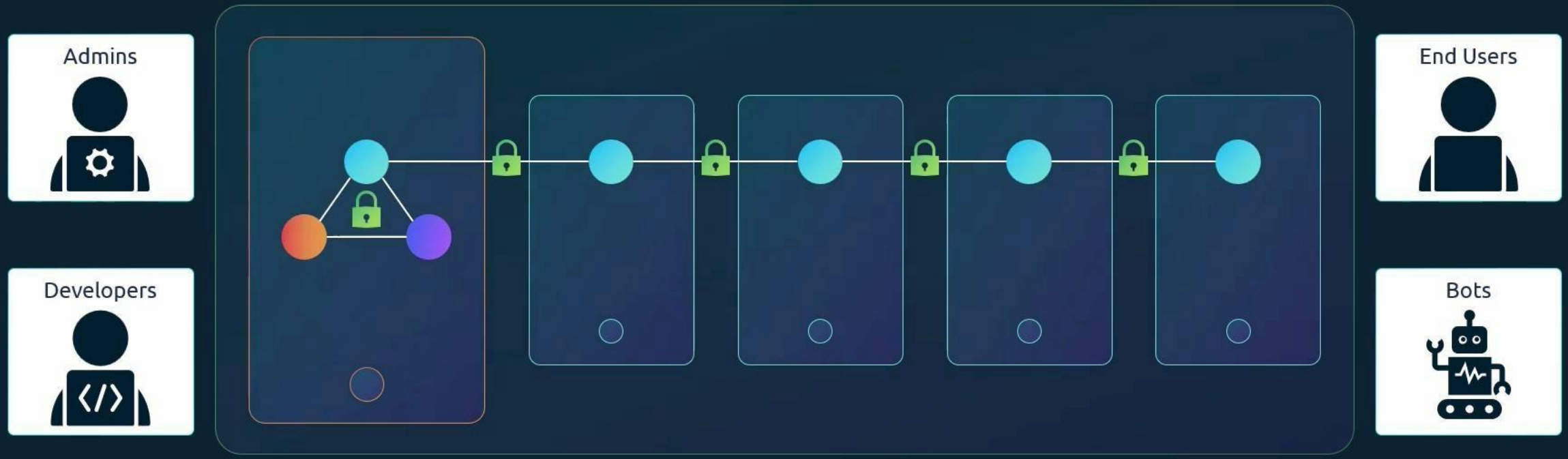


ASEGURANDO LA COMUNICACION ENTRE COMPONENTES

- Las comunicaciones seguras entre los componentes de Kubernetes se habilitan mediante cifrado TLS. Esto garantiza que los datos transmitidos entre los componentes clave se mantengan confidenciales y protegidos contra manipulaciones. El cifrado protege:
  - La comunicación dentro del clúster de etcd
  - Las interacciones entre el Kube Controller Manager y el Kube Scheduler
  - Las conexiones entre los componentes de los nodos de trabajo, como el Kubelet y el Kube Proxy

Network Policies





© Copyright KodeKloud

Auth Mechanisms

kube-apiserver

Static Token File

Certificates

Identity Services

© Copyright KodeKloud

Mechanism	Use case	Example / Notes
Static basic-auth file	Small experiments, local demos	CSV of username/password (plaintext); configured via —basic-auth-file (deprecated)
Static token file	Simple automation, throwaway clusters	CSV mapping bearer tokens to users; configured via —token-auth-file (deprecated)
TLS client certificates	Secure machine access, admin/user certificates	Use CA-signed client certs; verified by kube-apiserver
External identity providers (OIDC, LDAP)	Enterprise SSO, centralized user management	Integrate kube-apiserver with OIDC/LDAP for federated auth

Basic-auth CSV

```
# password,username,uid,groups
KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B,user10,u0010,group1
rJjncHmvtXHc6MlWQddhtvNyvhgTdxSC,user11,u0011,group1
mjp0FIEiFOKL9toikaRNtt59ePtczZSq,user12,u0012,group2
PG41IXhs7QjqwWkmBkvgGT9g10yUqZij,user13,u0013,group2
```

curl -k -u user10:KpjCVbI7rCFAHYPkByTIzRb7gu1cUc4B https://master-node-ip:6443/api

TOKEN-auth CSV

```
# token,username,uid,groups
c29tZXRva2VuMTIz,user21,u0021,group1
aW5vdGhlcnRva2Vu,user22,u0022,group2
```

curl -k -H "Authorization: Bearer c29tZXRva2VuMTIz" https://master-node-ip:6443/api

User: John
Password: Pass123

User: John
Password: Pass123

http://my-bank.com

ASYMMETRIC ENCRYPTION

Private Key

Public Lock

User: John
Password: Pass123

Certificate (Public Key)

---

*.crt *.pem

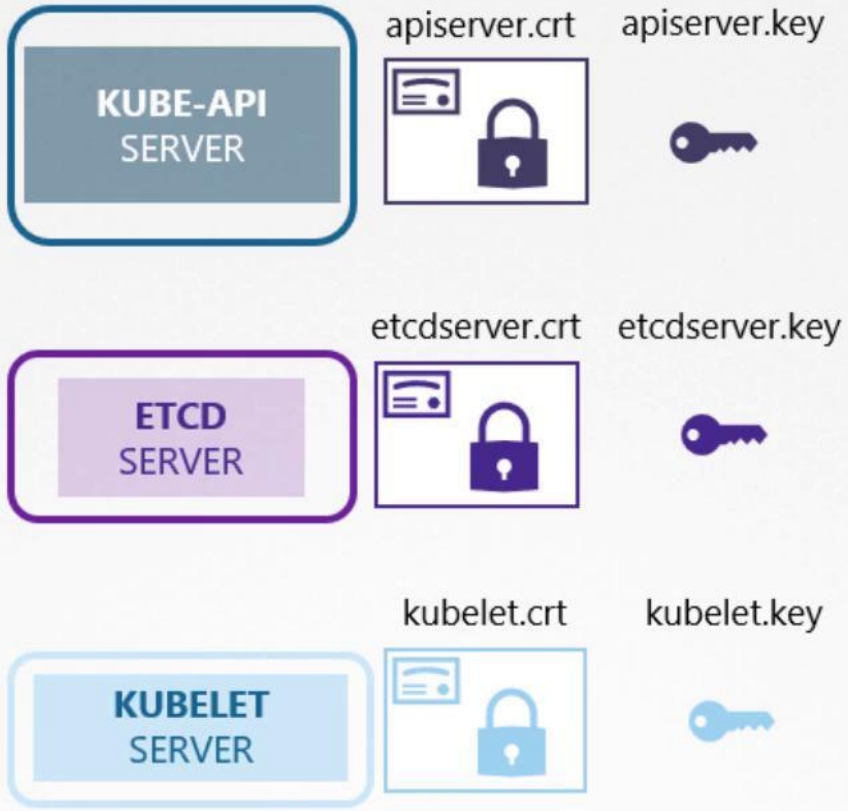
server.crt
server.pem
client.crt
client.pem

Public Key (Lock)

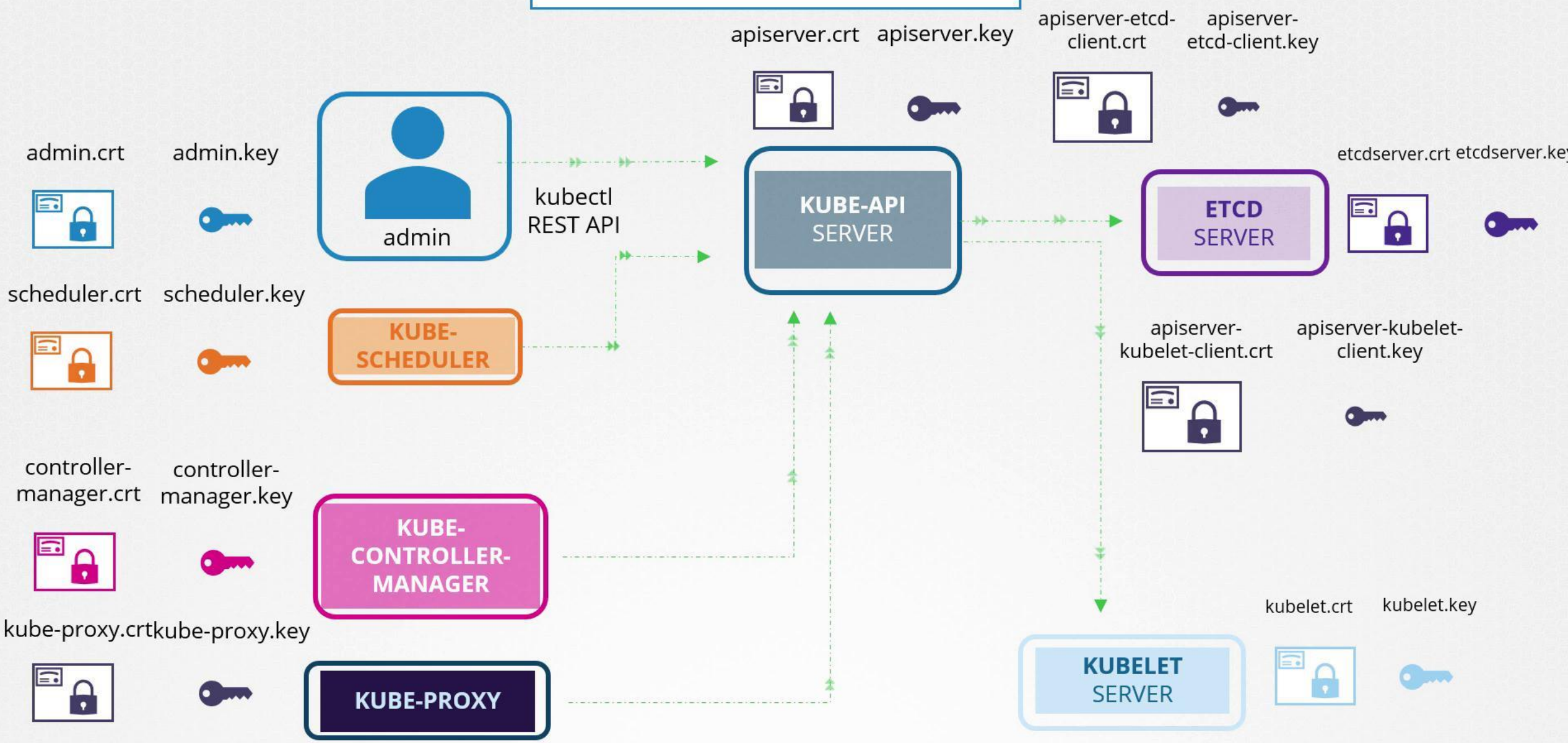
Private Key

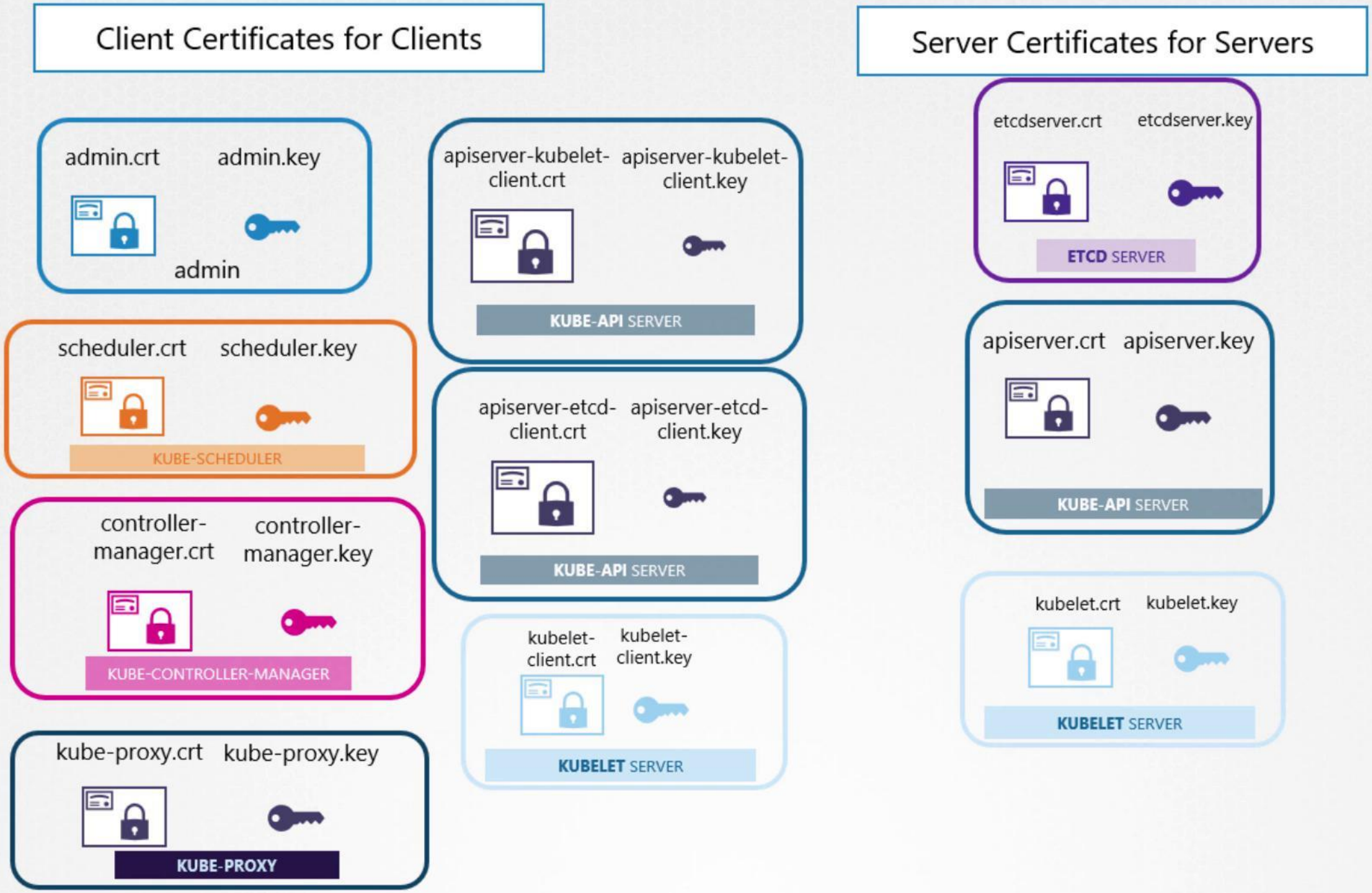
XCVB: DKSJD
LKJSDFK: XZKJSDLF

Server Certificates for Servers



Client Certificates for Clients




Client Certificates for Clients
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
KUBE-API SERVER
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
Server Certificates for Servers
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet.crt
kubelet.key
KUBELET SERVER

Generar el CA Certificate

```
openssl genrsa -out ca.key 2048
openssl req -new -key ca.key -subj "/CN=KUBERNETES-CA" -out ca.csr
openssl x509 -req -in ca.csr -signkey ca.key -out ca.crt
```

Generar el CA Certificate

```
openssl genrsa -out admin.key 2048
openssl req -new -key admin.key -subj "/CN=kube-admin/O=system:masters" -
out admin.csr
openssl x509 -req -in admin.csr -CA ca.crt -CAkey ca.key -out admin.crt
```

KUBE SCHEDULER

ca.key

ca.crt

Generate Keys

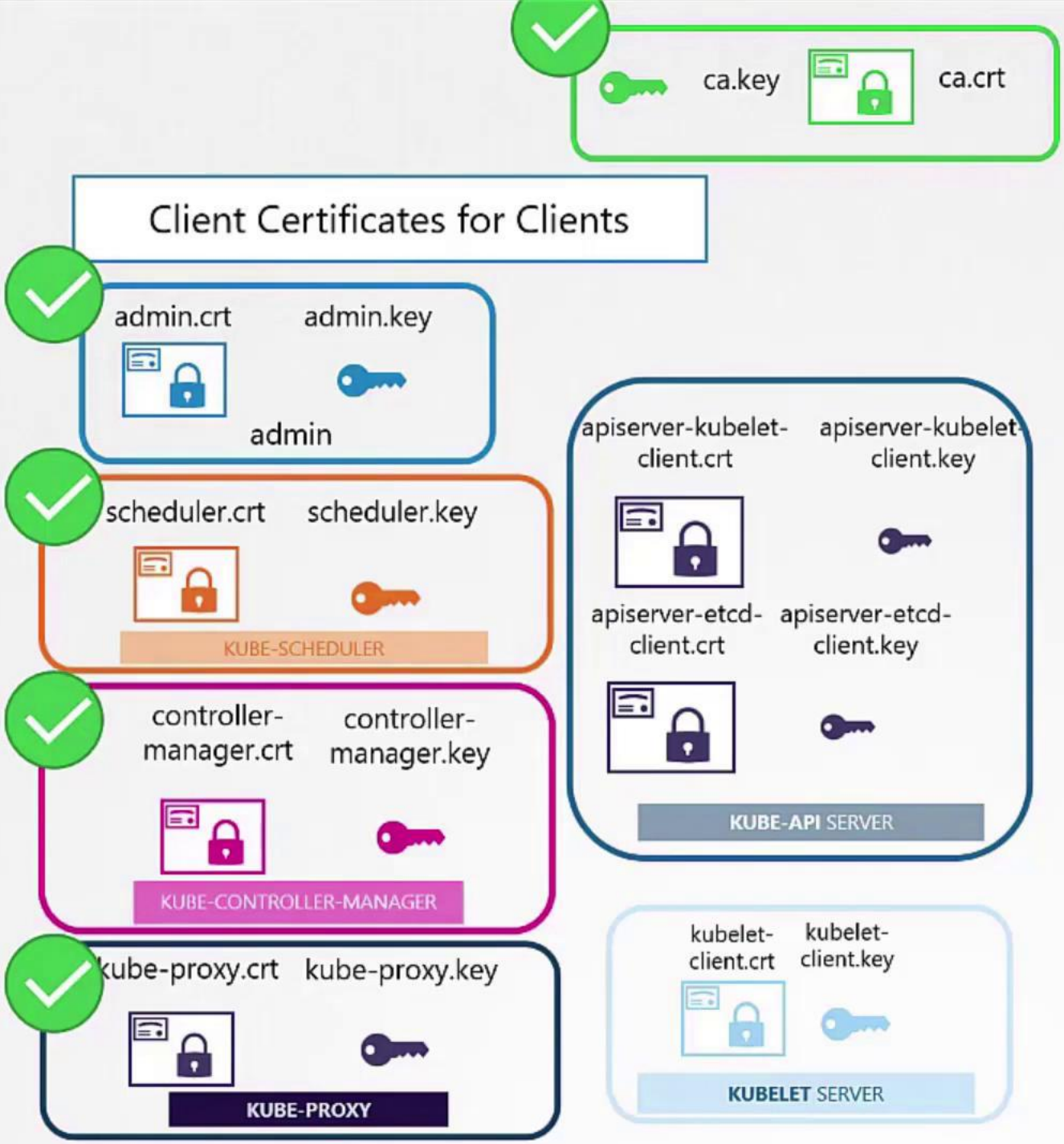
scheduler.key

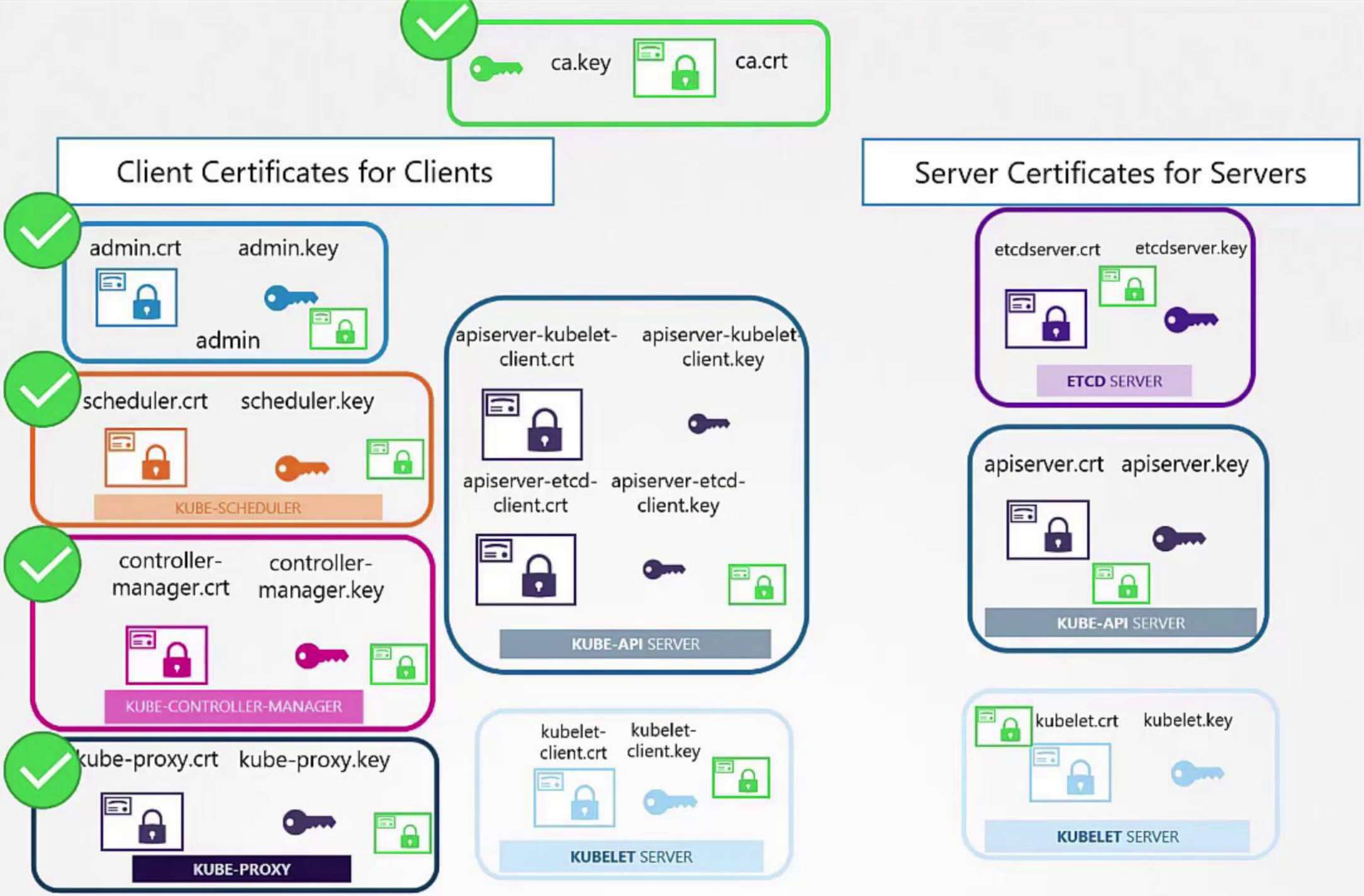
Certificate Signing Request

scheduler.csr

Sign Certificates

scheduler.crt


ca.key
ca.crt
Client Certificates for Clients
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER


ca.key
ca.crt
Client Certificates for Clients
admin.crt
admin.key
admin
scheduler.crt
scheduler.key
KUBE-SCHEDULER
controller-manager.crt
controller-manager.key
KUBE-CONTROLLER-MANAGER
kube-proxy.crt
kube-proxy.key
KUBE-PROXY
apiserver-kubelet-client.crt
apiserver-kubelet-client.key
apiserver-etcd-client.crt
apiserver-etcd-client.key
KUBE-API SERVER
kubelet-client.crt
kubelet-client.key
KUBELET SERVER
Server Certificates for Servers
etcdserver.crt
etcdserver.key
ETCD SERVER
apiserver.crt
apiserver.key
KUBE-API SERVER
kubelet.crt
kubelet.key
KUBELET SERVER

```
cat etcd.yaml
- --advertise-client-urls=https://127.0.0.1:2379
- --key-file=/path-to-certs/etcdserver.key
- --cert-file=/path-to-certs/etcdserver.crt
- --client-cert-auth=true
- --data-dir=/var/lib/etcd
- --initial-advertise-peer-urls=https://127.0.0.1:2380
- --initial-cluster=master=https://127.0.0.1:2380
- --listen-client-urls=https://127.0.0.1:2379
- --listen-peer-urls=https://127.0.0.1:2380
- --name=master
- --peer-cert-file=/path-to-certs/etcdpeer1.crt
- --peer-client-cert-auth=true
- --peer-key-file=/etc/kubernetes/pki/etcd/peer.key
- --peer-trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
- --snapshot-count=10000
- --trusted-ca-file=/etc/kubernetes/pki/etcd/ca.crt
```

API Server Certificate

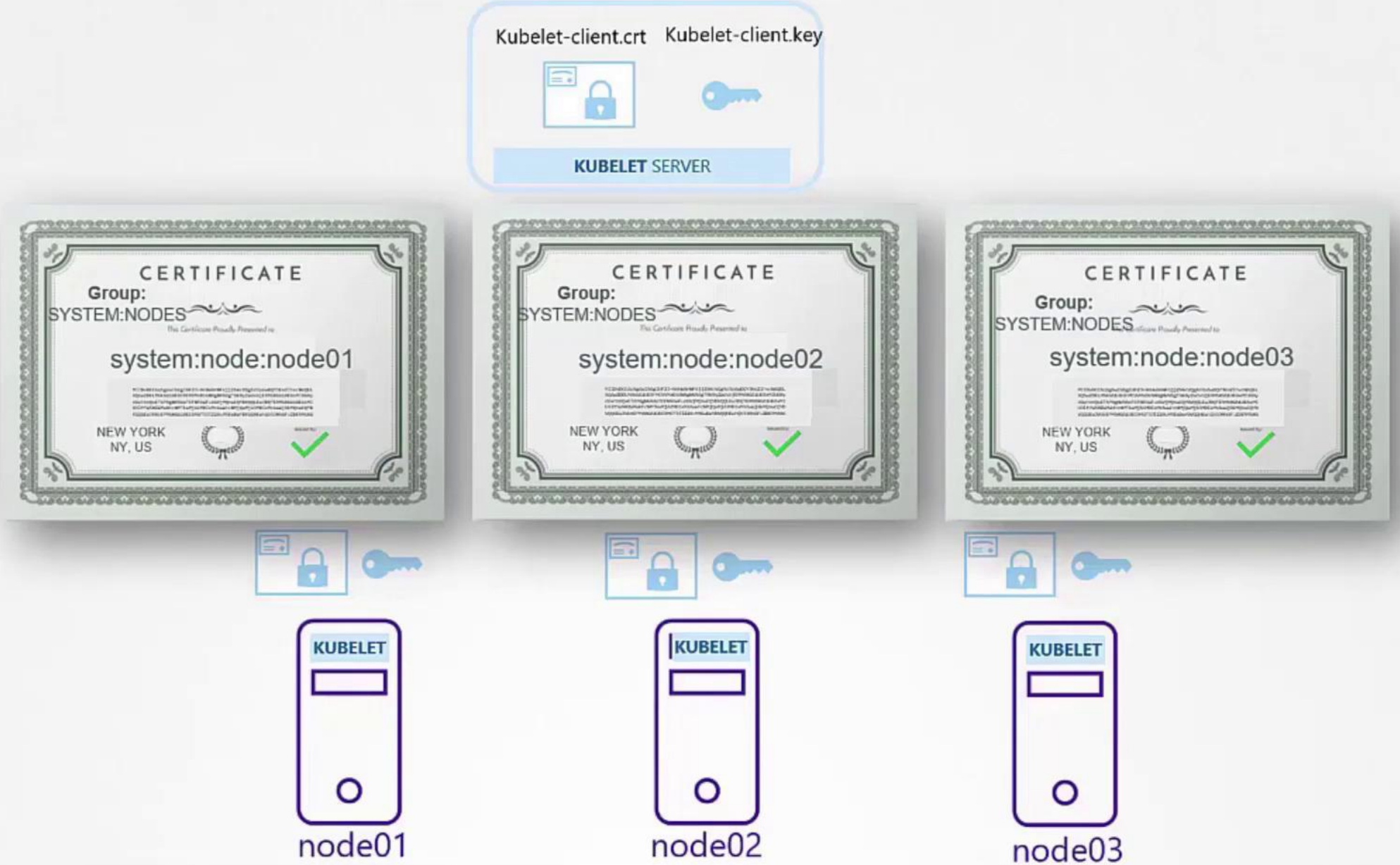
```
openssl req -new -key apiserver.key -subj "/CN=kube-apiserver" -out apiserver.csr
```

```
[req]
req_extensions = v3_req
distinguished_name = req_distinguished_name

[v3_req]
basicConstraints = CA:FALSE
keyUsage = nonRepudiation, digitalSignature, keyEncipherment
subjectAltName = @alt_names

[alt_names]
DNS.1 = kubernetes
DNS.2 = kubernetes.default
DNS.3 = kubernetes.default.svc
DNS.4 = kubernetes.default.svc.cluster.local
IP.1 = 10.96.0.1
IP.2 = 172.17.0.87
```

openssl.cnf

```
ExecStart=/usr/local/bin/kube-apiserver \
  --advertise-address=${INTERNAL_IP} \
  --allow-privileged=true \
  --apiserver-count=3 \
  --authorization-mode=Node,RBAC \
  --bind-address=0.0.0.0 \
  --enable-swagger-ui=true \
  --etcd-cafile=/var/lib/kubernetes/ca.pem \
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt \
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key \
  --etcd-servers=https://127.0.0.1:2379 \
  --event-ttl=1h \
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \
  --kubelet-client-certificate=/var/lib/kubernetes/apiserver-kubelet-client.crt \
  --kubelet-client-key=/var/lib/kubernetes/apiserver-kubelet-client.key \
  --kubelet-https=true \
  --runtime-config=api/all \
  --service-account-key-file=/var/lib/kubernetes/service-account.pem \
  --service-cluster-ip-range=10.32.0.0/24 \
  --service-node-port-range=30000-32767 \
  --client-ca-file=/var/lib/kubernetes/ca.pem \
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \
  --v=2
```

KUBECTL NODES (CLIENT CERT)



Autenticación vía curl y kubectl

```
curl https://my-kube-playground:6443/api/v1/pods \
  --key admin.key \
  --cert admin.crt \
  --cacert ca.crt
```


```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods"
  },
  "items": []
}
```

Autenticación vía curl y kubectl

```
kubectl get pods \
  --server https://my-kube-playground:6443 \
  --client-key admin.key \
  --client-certificate admin.crt \
  --certificate-authority ca.crt
```


```
{
  "kind": "PodList",
  "apiVersion": "v1",
  "metadata": {
    "selfLink": "/api/v1/pods"
  },
  "items": []
}
```

Kubeconfig.yaml

```yaml
apiVersion: v1
kind: Config
clusters:
- name: my-kube-playground  # values hidden…
- name: development
- name: production
- name: google
contexts:
- name: my-kube-admin@my-kube-playground
- name: dev-user@google
- name: prod-user@production
users:
- name: my-kube-admin
- name: admin
- name: dev-user
- name: prod-user
```

kubectl get pods

kubectl config view #te muestra el kubeconfig

kubectl config view

```yaml
apiVersion: v1
kind: Config
current-context: kubernetes-admin@kubernetes
clusters:
- cluster:
    certificate-authority-data: REDACTED
    server: https://172.17.0.5:6443
  name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
users:
- name: kubernetes-admin
  user:
    client-certificate-data: REDACTED
    client-key-data: REDACTED
```

kubectl config use-context prod-user@production

```yaml
apiVersion: v1
kind: Config
current-context: my-kube-admin@my-kube-playgroun
clusters:
- name: my-kube-playground
- name: development
- name: production
contexts:
- name: my-kube-admin@my-kube-playground
- name: prod-user@production
users:
- name: my-kube-admin
- name: prod-user
```

kubectl config use-context prod-user@production

```yaml
apiVersion: v1
kind: Config
clusters:
- name: production
  cluster:
    certificate-authority: ca.crt
    server: https://172.17.0.51:6443
contexts:
- name: admin@production
  context:
    cluster: production
    user: admin
    namespace: finance
users:
- name: admin
  user:
    client-certificate: admin.crt
    client-key: admin.key
```

/metrics

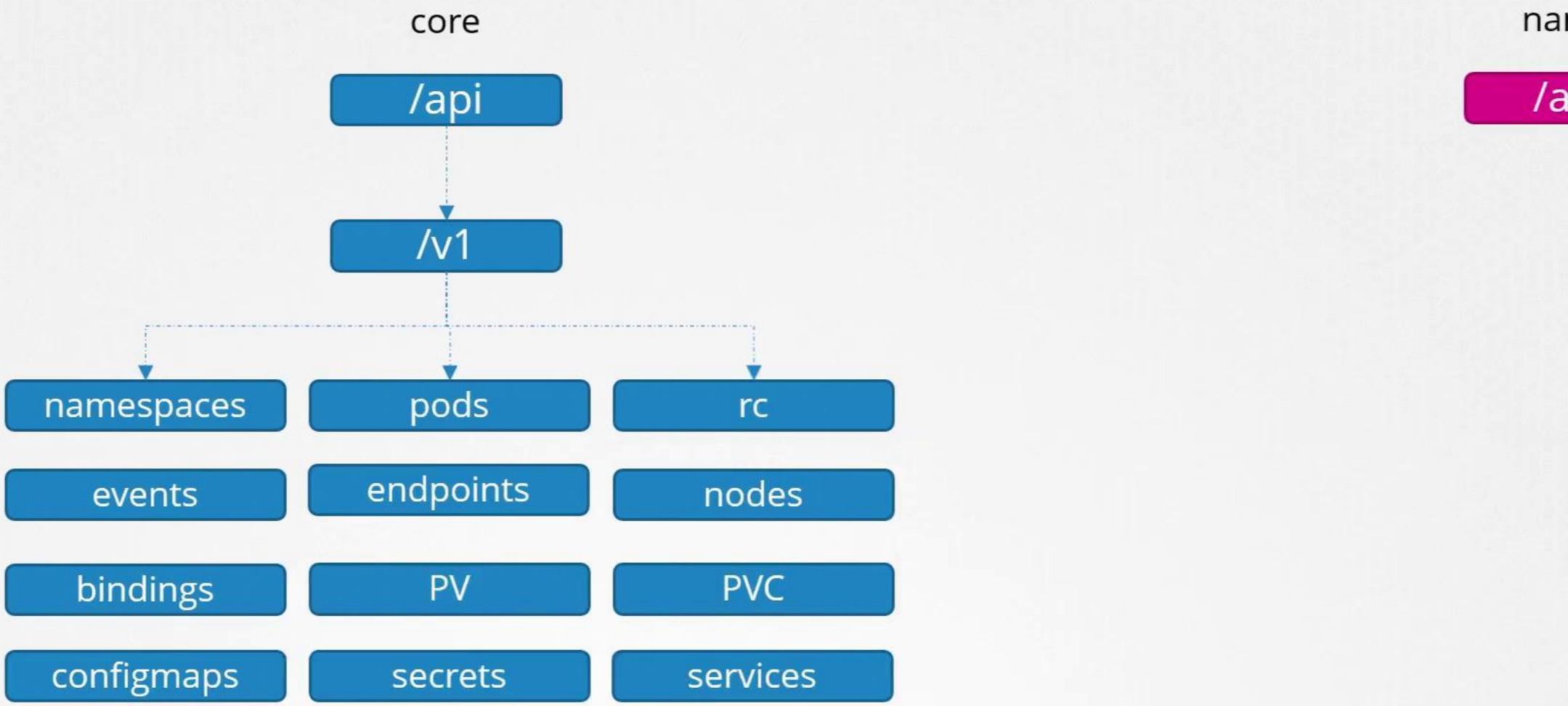
/healthz

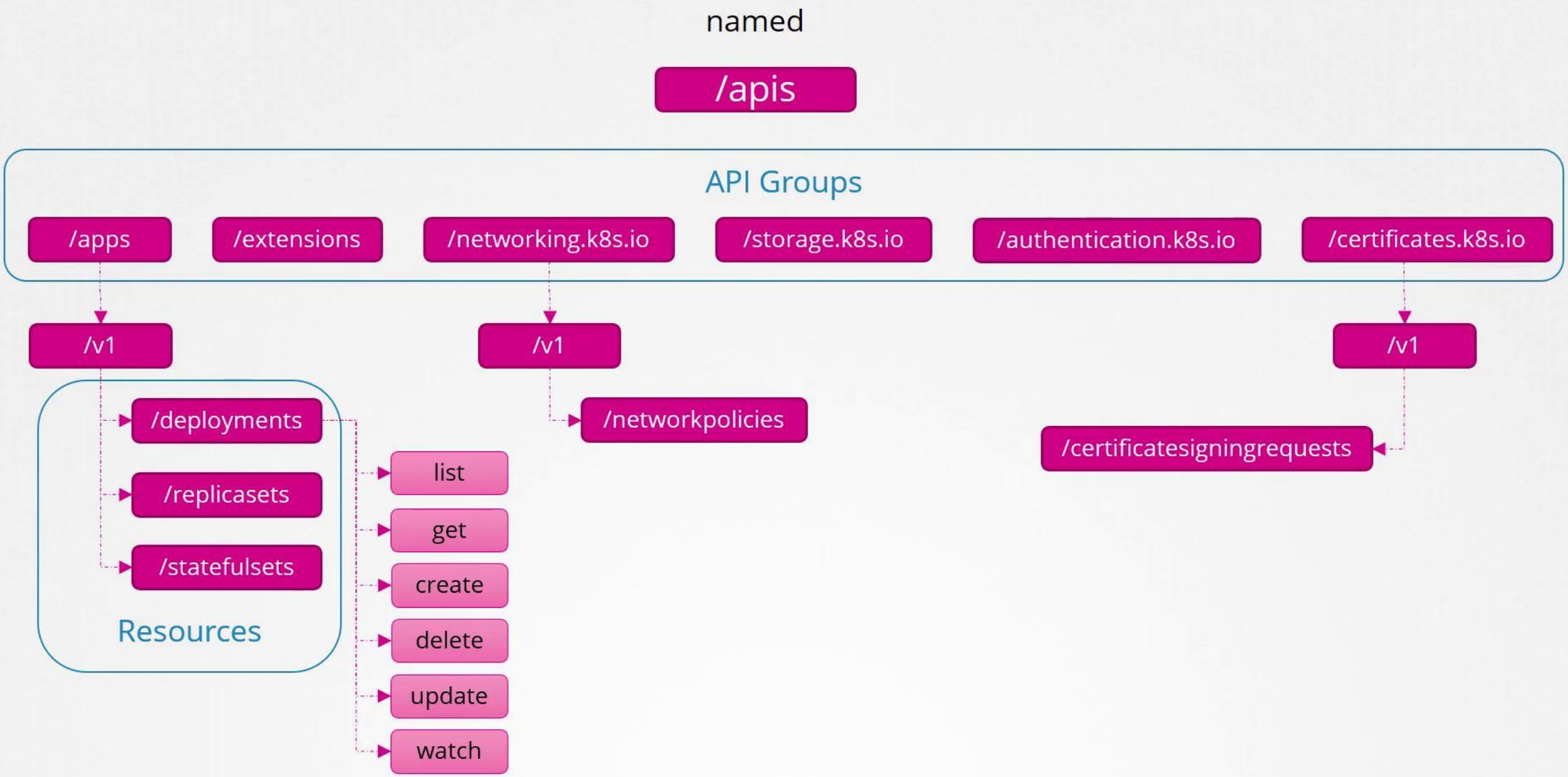
/version

/api

/apis

/logs


core
/api
/v1
namespaces
pods
rc
events
endpoints
nodes
bindings
PV
PVC
configmaps
secrets
services
na
/a


named
/apis
API Groups
/apps
/extensions
/networking.k8s.io
/storage.k8s.io
/authentication.k8s.io
/certificates.k8s.io
/v1
/v1
/v1
/deployments
/replicasets
/statefulsets
Resources
list
get
create
delete
update
watch
/networkpolicies
/certificatesigningrequests

Consultar el API Server

```
curl http://localhost:6443 -k
```


```
{
  "paths": [
    "/api",
    "/api/v1",
    "/apis",
    "/apis/",
    "/healthz",
    "/logs",
    "/metrics",
    "/openapi/v2",
    "/swagger-2.0.0.json"
  ]
}
```

Authorization

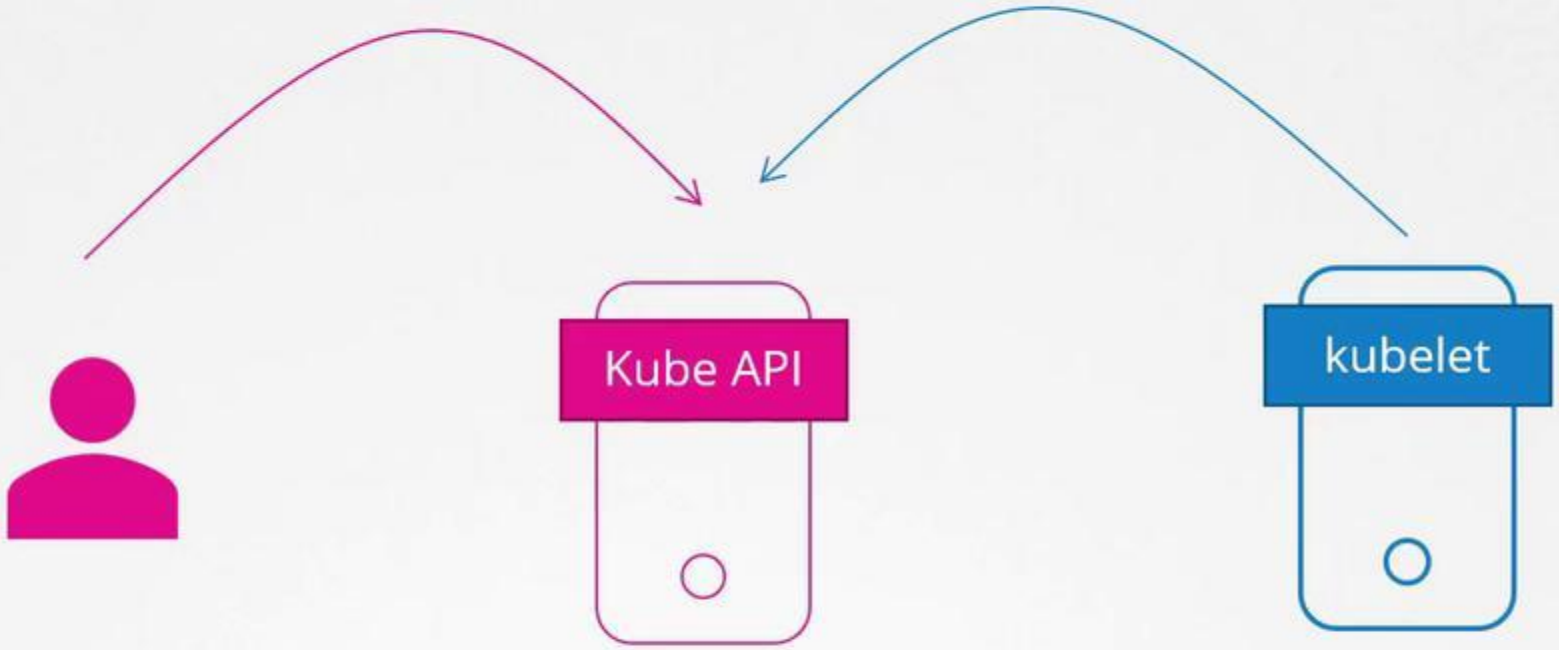
```
kubectl get pods
NAME     READY   STATUS     RESTARTS   AGE
nginx    1/1     Running    0          53s

kubectl get nodes
NAME         STATUS   ROLES      AGE      VERSION
worker-1     Ready    <none>     5d21h    v1.13.0
worker-2     Ready    <none>     5d21h    v1.13.0
```

```
kubectl get pods
Error from server (Forbidden): pods is forbidden: User "Bot-1" cannot list "pods"

kubectl get nodes
Error from server (Forbidden): nodes is forbidden: User "Bot-1" cannot get "nodes"

kubectl delete node worker-2
Error from server (Forbidden): nodes "worker-2" is forbidden: User "developer" cannot
delete resource "nodes"
```

Mecanismos de Authorization K8S

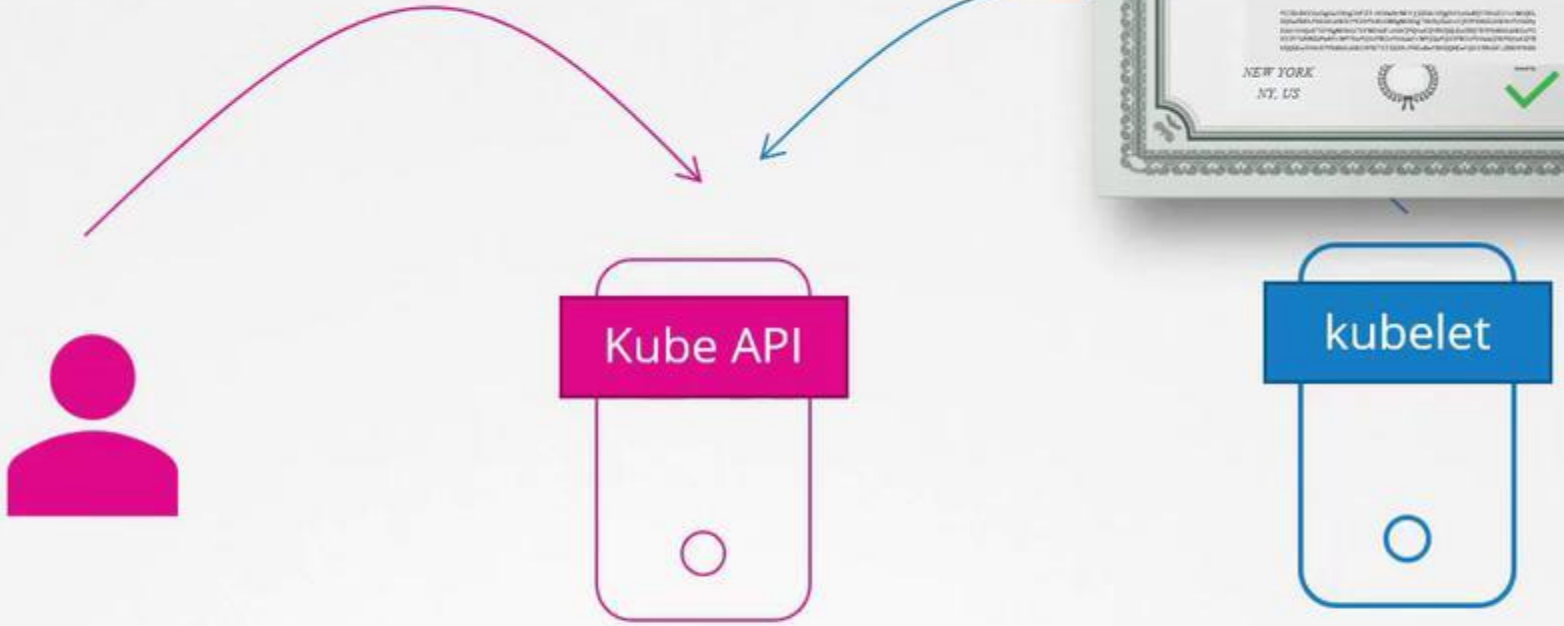
- Kubernetes admite múltiples mecanismos de autorización, entre ellos:
  - Autorización de Nodos (Node Authorization)
  - Autorización basada en atributos (Attribute-Based Authorization - ABAC)
  - Control de acceso basado en roles (Role-Based Access Control - RBAC)
  - Autorización mediante Webhook (Webhook Authorization)

Node



- Read
  - Services
  - Endpoints
  - Nodes
  - Pods
- Write
  - Node status
  - Pod status
  - events

Node Authorizer

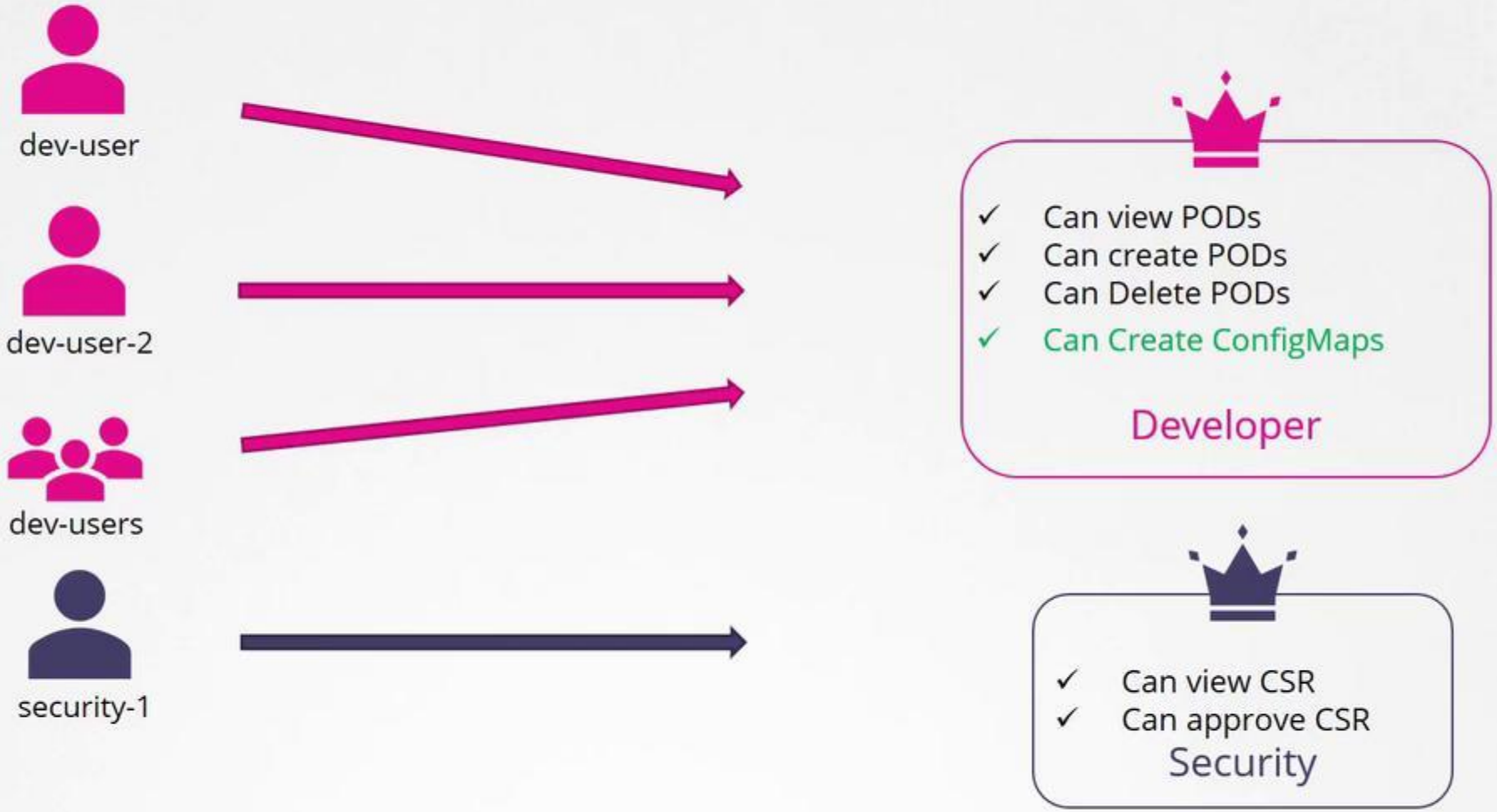


- Read
  - Services
  - Endpoints
  - Nodes
  - Pods
- Write
  - Node status
  - Pod status
  - events

AUTHORIZATION BASADA EN ATRIBUTOS

```
{"kind": "Policy", "spec": {"user": "dev-user", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "dev-user-2", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"group": "dev-users", "namespace": "*", "resource": "pods", "apiGroup": "*"}}
{"kind": "Policy", "spec": {"user": "security-1", "namespace": "*", "resource": "csr", "apiGroup": "*"}}
```

RBAC


dev-user
dev-user-2
dev-users
security-1
✓ Can view PODs
✓ Can create PODs
✓ Can Delete PODs
✓ Can Create ConfigMaps
Developer
✓ Can view CSR
✓ Can approve CSR
Security

Modos de Authorization que soporta K8S

- **AlwaysAllow**: Permite todas las solicitudes sin realizar ninguna verificación de autorización.
- **AlwaysDeny**: Deniega todas las solicitudes.

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=AlwaysAllow \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.pem \\
  --kubelet-client-certificate=/var/lib/kubernetes/apiserver-et
  --kubelet-client-key=/var/lib/kubernetes/apiserver-etcd-clien
  --service-node-port-range=30000-32767 \\
  --client-ca-file=/var/lib/kubernetes/ca.pem \\
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \\
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \\
  -v=2
```

```
ExecStart=/usr/local/bin/kube-apiserver \\
  --advertise-address=${INTERNAL_IP} \\
  --allow-privileged=true \\
  --apiserver-count=3 \\
  --authorization-mode=Node,RBAC,Webhook \\
  --bind-address=0.0.0.0 \\
  --enable-swagger-ui=true \\
  --etcd-cafile=/var/lib/kubernetes/ca.pem \\
  --etcd-certfile=/var/lib/kubernetes/apiserver-etcd-client.crt
  --etcd-keyfile=/var/lib/kubernetes/apiserver-etcd-client.key
  --etcd-servers=https://127.0.0.1:2379 \\
  --event-ttl=1h \\
  --kubelet-certificate-authority=/var/lib/kubernetes/ca.crt \\
  --tls-cert-file=/var/lib/kubernetes/apiserver.crt \\
  --tls-private-key-file=/var/lib/kubernetes/apiserver.key \\
  --v=2
```

CREANDO UN ROLE

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["list", "get", "create", "update", "delete"]
- apiGroups: [""]
  resources: ["ConfigMap"]
  verbs: ["create"]
```

kubectl create -f developer-role.yaml

ASOCIAR UN ROLE

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: devuser-developer-binding
subjects:
- kind: User
  name: dev-user
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: developer
  apiGroup: rbac.authorization.k8s.io
```

kubectl create -f devuser-developer-binding.yaml

```
kubectl get roles

kubectl get rolebindings

kubectl describe role developer
```

```
Name:         developer
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources         Non-Resource URLs   Resource Names   Verbs
  -----------       ------------------  --------------   ----
  ConfigMap         []                  []               [create]
  pods              []                  []               [get watch list create delete]
```

Kubectl auth

kubectl auth can-i create deployments

Yes|no

kubectl auth can-i delete nodes

```
kubectl auth can-i create deployments
# Output: yes
kubectl auth can-i delete nodes
# Output: no
kubectl auth can-i create deployments --as dev-user
# Output: no
kubectl auth can-i create pods --as dev-user
# Output: yes
```

Limitar acceso a recursos específicos

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
```

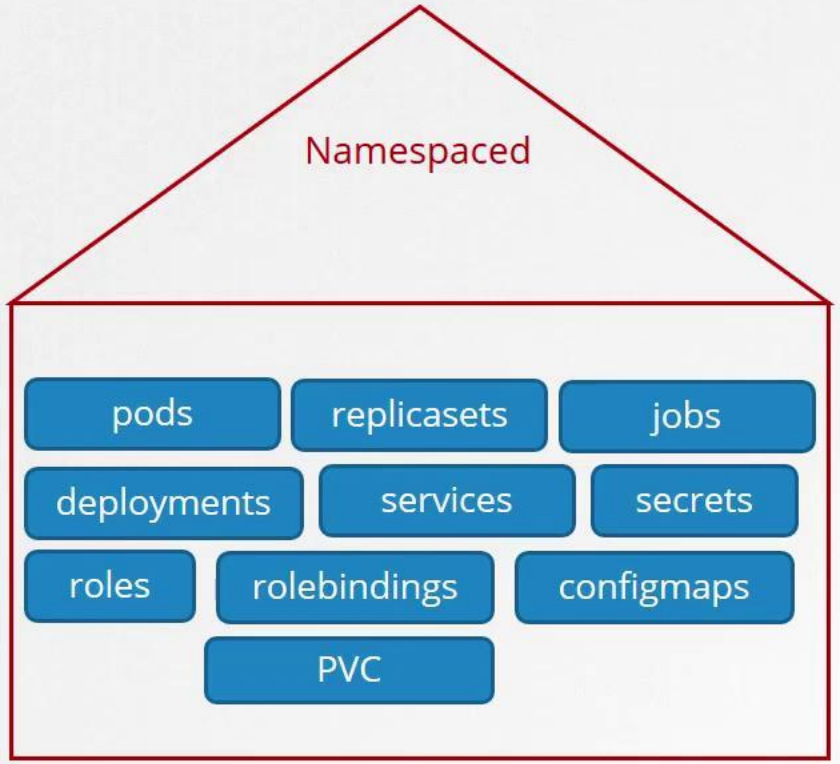
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: developer
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "create", "update"]
  resourceNames: ["blue", "orange"]
```

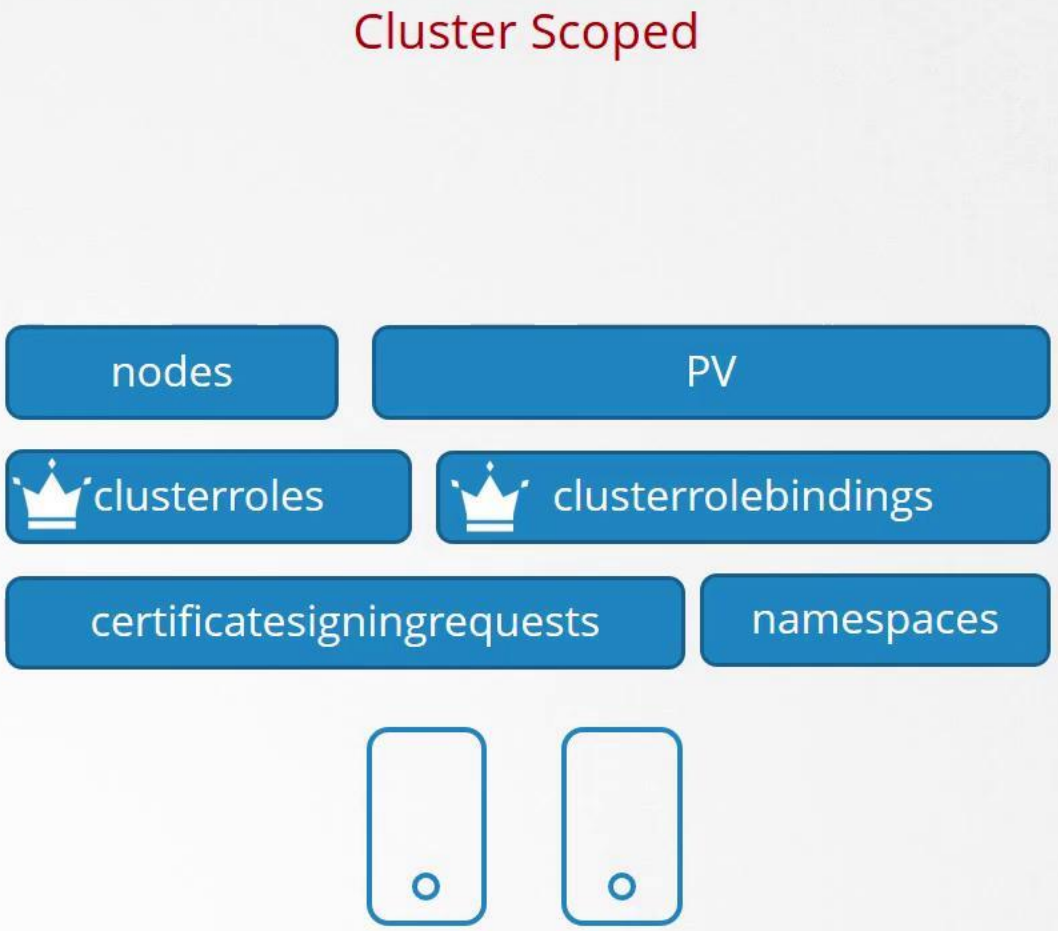
Cluster Roles y Cluster Role Bindings

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cluster-administrator
rules:
- apiGroups: [""]
  resources: ["nodes"]
  verbs: ["list", "get", "create", "delete"]
```

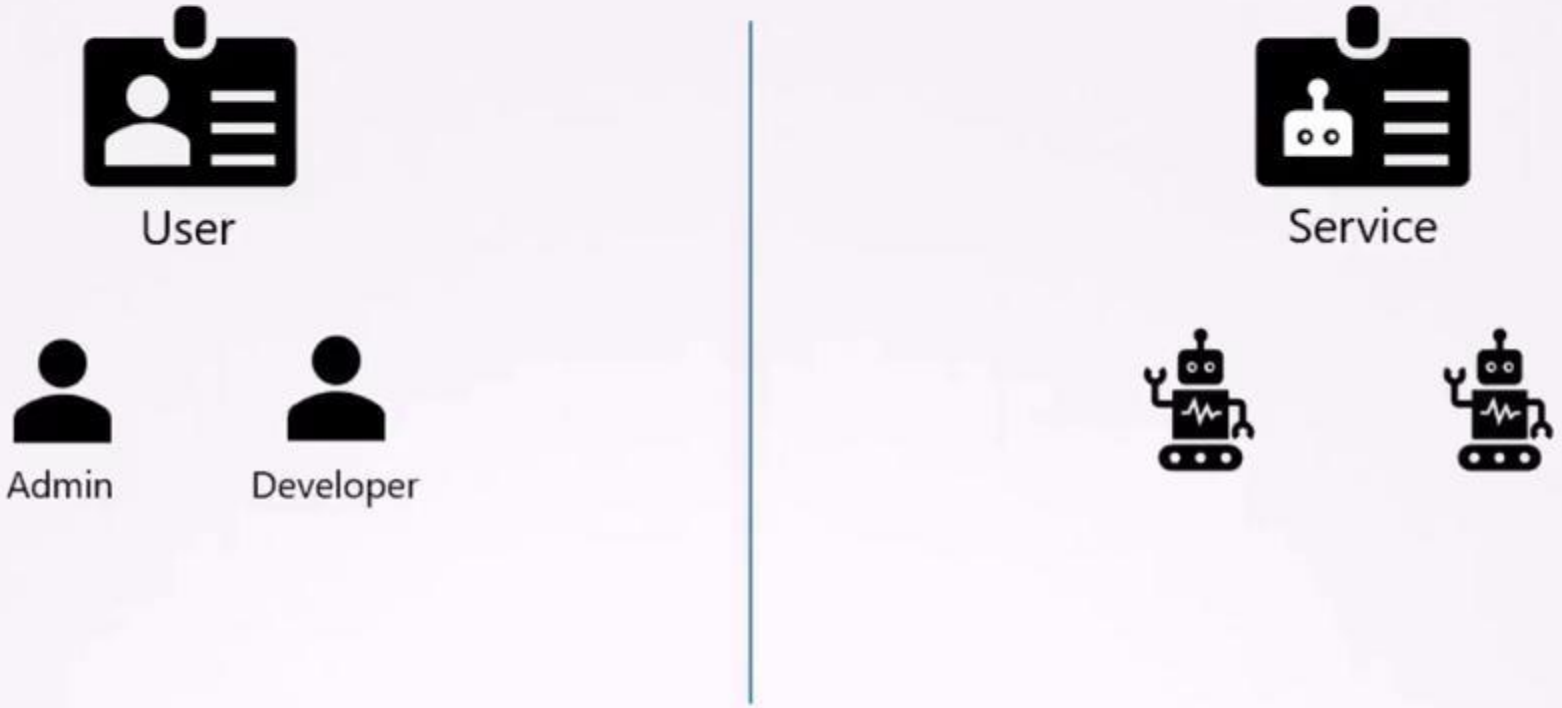
```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: cluster-admin-role-binding
subjects:
- kind: User
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: ClusterRole
  name: cluster-administrator
  apiGroup: rbac.authorization.k8s.io
```

Cluster Roles

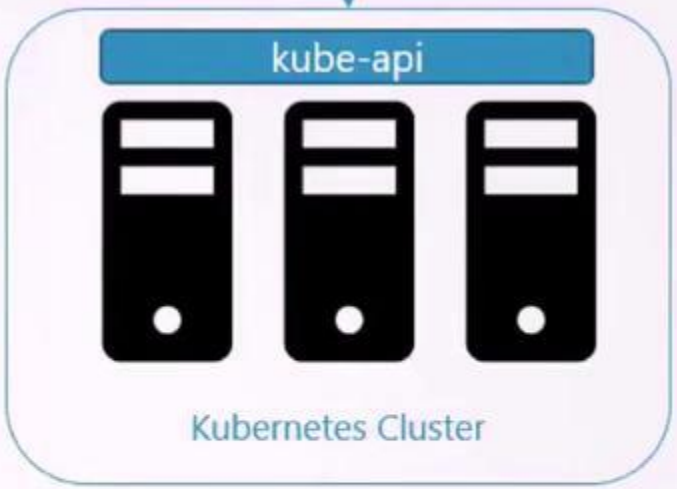




Service Accounts



My Kubernetes Dashboard
LOAD DASHBOARD
Name
State
Containers
Service Account
IP


kube-api
Kubernetes Cluster

```
kubectl create serviceaccount dashboard-sa
```

```
kubectl get serviceaccount
```

```
NAME           SECRETS   AGE
default        1         218d
dashboard-sa   1         4d
```

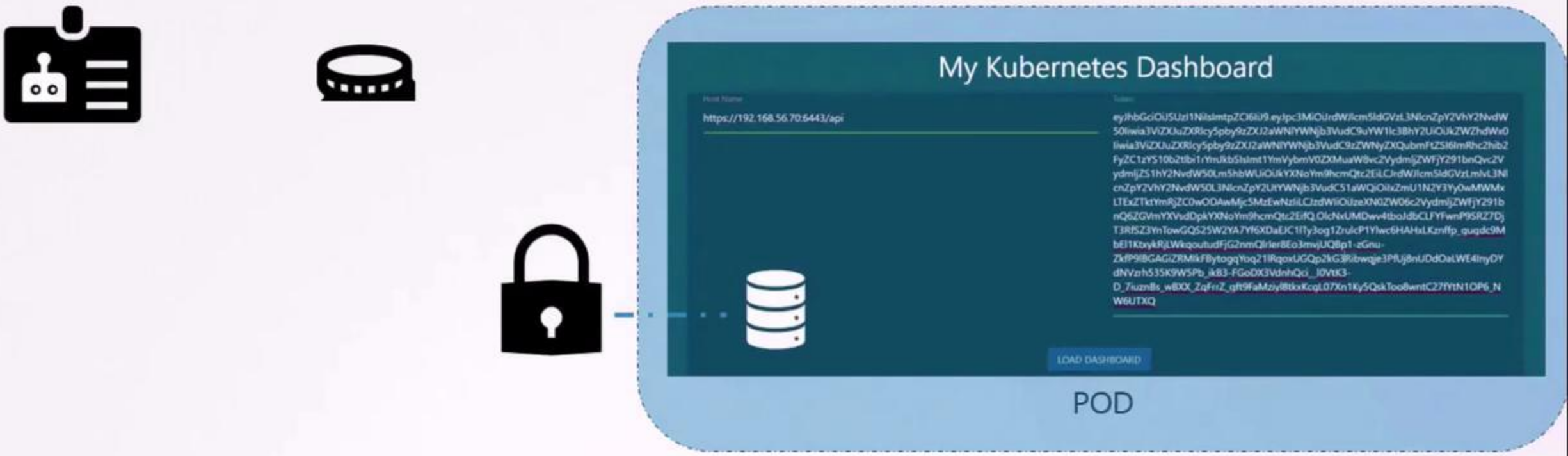
kubectl describe serviceaccount dashboard-sa

```
Name:                dashboard-sa
Namespace:           default
Labels:              <none>
Annotations:         <none>
Image pull secrets:  <none>
Mountable secrets:   dashboard-sa-token-kbbdm
Tokens:              dashboard-sa-token-kbbdm
Events:              <none>
```

kubectl describe secret dashboard-sa-token-kbbdm

token: eyJhbGciOiJSUzI1NiIsImtpZCI6Ij... (truncated for privacy)

```
curl https://192.168.56.70:6443/api -k \
--header "Authorization: Bearer eyJhbgG..."
```



/var/run/secrets/kubernetes.io/serviceaccount

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-kubernetes-dashboard
spec:
  containers:
  - name: my-kubernetes-dashboard
    image: my-kubernetes-dashboard
```

kubectl describe pod my-kubernetes-dashboard

```
Name:           my-kubernetes-dashboard
Namespace:      default
Status:         Running
IP:             10.244.0.15
Containers:
  nginx:
    Image:        my-kubernetes-dashboard
    Mounts:       /var/run/secrets/kubernetes.io/serviceaccoun
Volumes:
  default-token-j4hkv:
    Type:        Secret (a volume populated by a Secret)
    SecretName:  default-token-j4hkv
    Optional:    false
```

kubectl exec -it my-kubernetes-dashboard – ls /var/run/secrets/kubernetes.io/serviceaccount
kubectl exec -it my-kubernetes-dashboard – cat /var/run/secrets/kubernetes.io/serviceaccount/token

References

KEP 1205 - Bound Service Account Tokens

https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/1205-bound-service-account-tokens

KEP-2799: Reduction of Secret-based Service Account Tokens

https://github.com/kubernetes/enhancements/tree/master/keps/sig-auth/2799-reduction-of-secret-based-service-account-token

https://kubernetes.io/docs/concepts/configuration/secret/#service-account-token-secrets
https://kubernetes.io/docs/reference/access-authn-authz/service-accounts-admin/

Seguridad de Imagenes

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: nginx
```

docker.io/library/nginx

Tu-cuenta/nginx

Seguridad de Imagenes

docker login private-registry.io

Login with your Docker ID to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com to create one.
Username: registry-user
Password:
WARNING! Your password will be stored unencrypted in /home/vagrant/.docker/config.json.
Login Succeeded

Configurar Pods para registros privados

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
```

```bash
kubectl create secret docker-registry regcred \
  --docker-server=private-registry.io \
  --docker-username=registry-user \
  --docker-password=registry-password \
  --docker-email=registry-user@org.com
```

Configurar Pods para registros privados

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
spec:
  containers:
    - name: nginx
      image: private-registry.io/apps/internal-app
  imagePullSecrets:
    - name: regcred
```

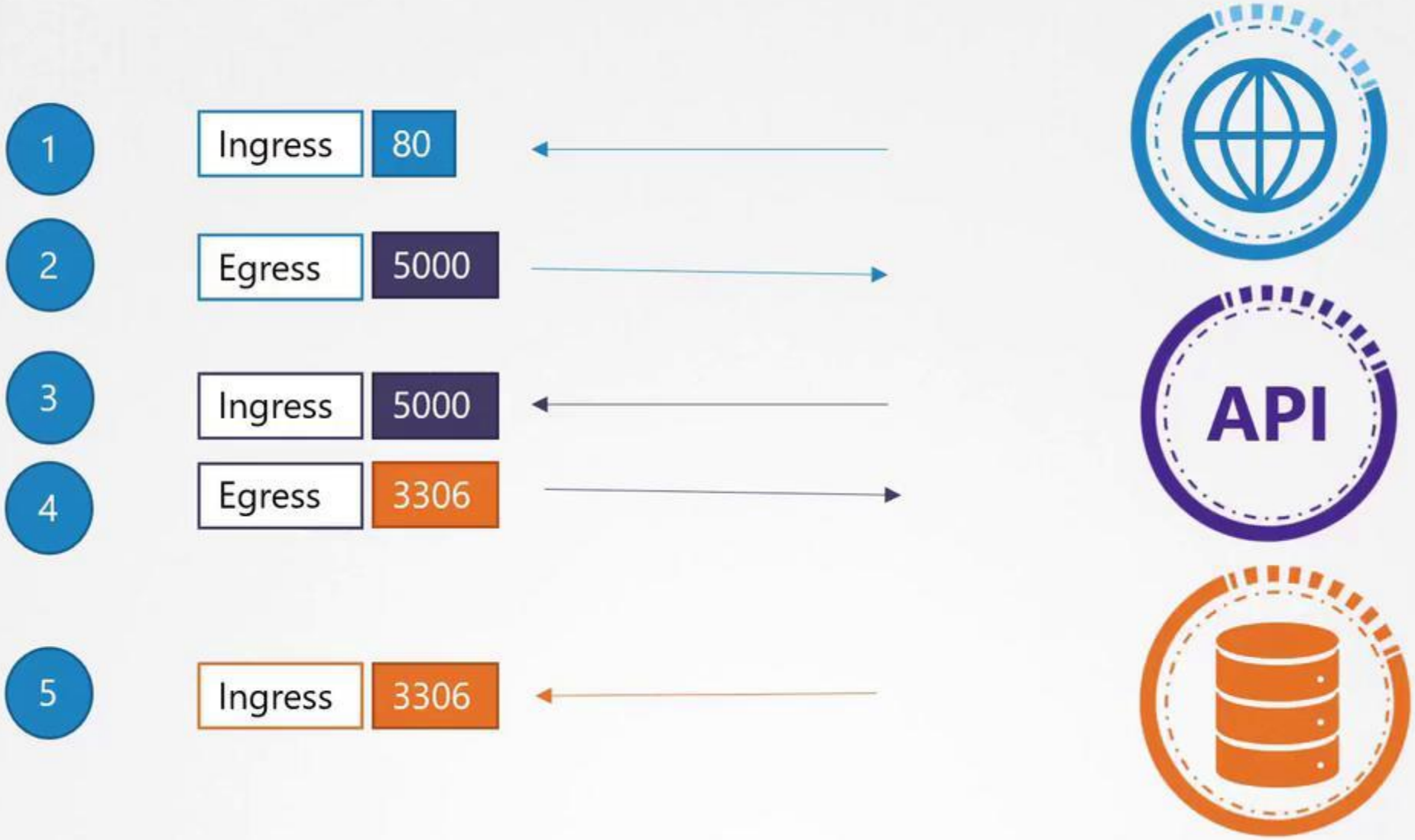
```bash
kubectl create secret docker-registry regcred \
  --docker-server=private-registry.io \
  --docker-username=registry-user \
  --docker-password=registry-password \
  --docker-email=registry-user@org.com
```

Contexto de seguridad a nivel de contenedor vs a nivel de Pod

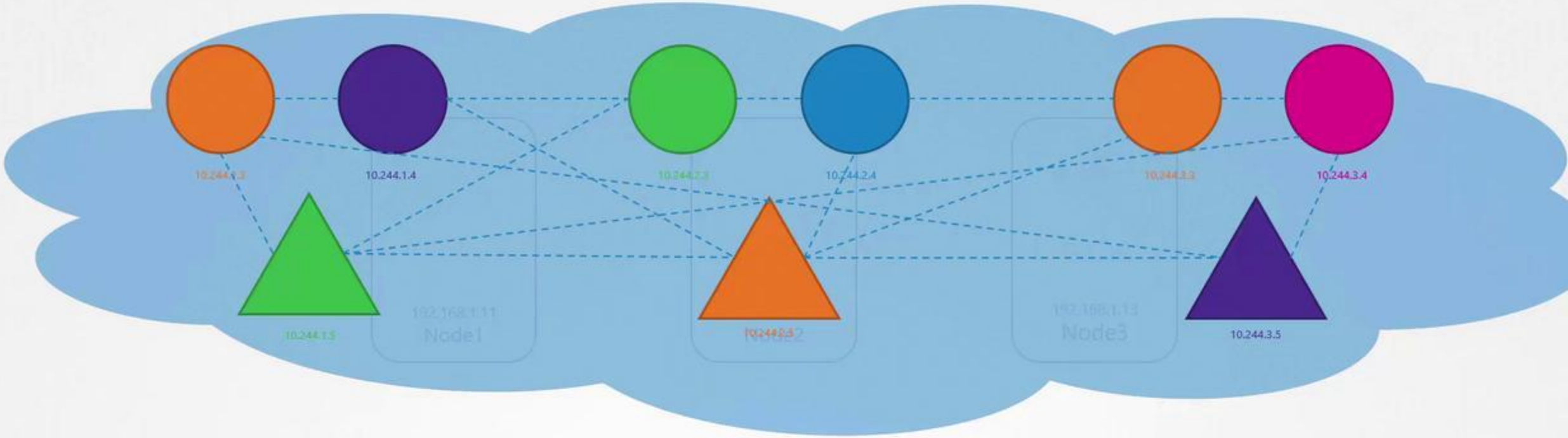
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        runAsUser: 1000
        capabilities:
          add: ["MAC_ADMIN"]
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: web-pod
spec:
  securityContext:
    runAsUser: 1000
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep", "3600"]
      securityContext:
        capabilities:
          add: ["MAC_ADMIN"]
```

Traffic


1
Ingress
80
2
Egress
5000
3
Ingress
5000
4
Egress
3306
5
Ingress
3306
API

Network Security

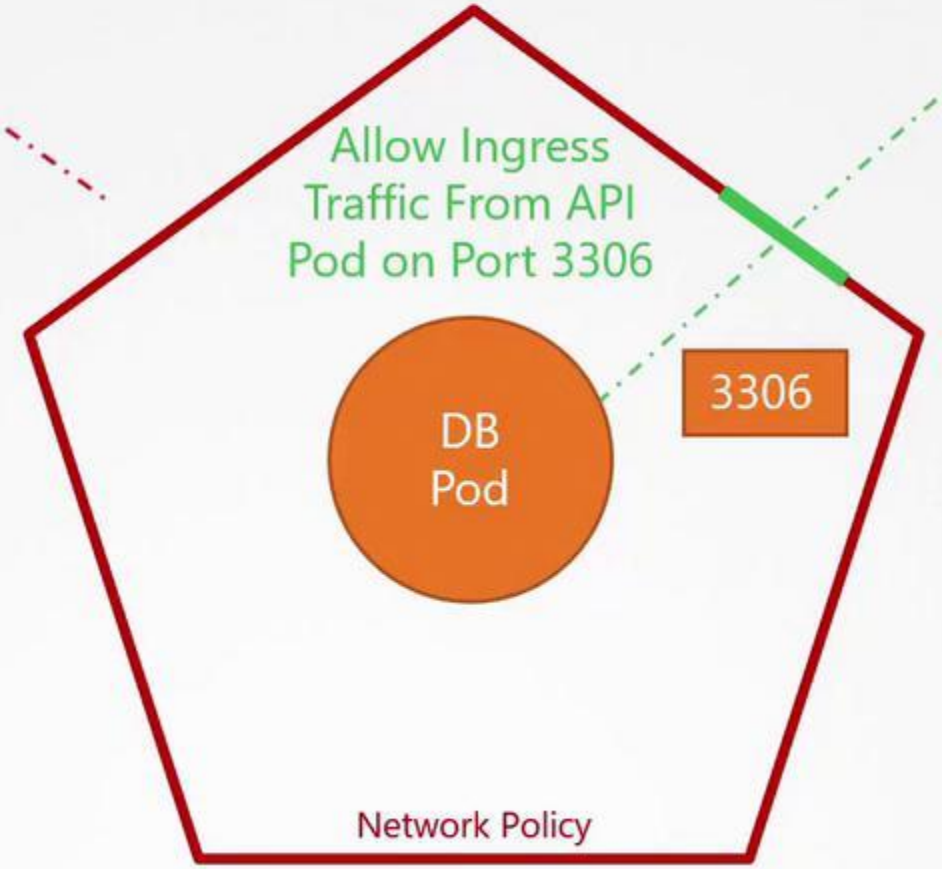


"All Allow"

Network Policy

Network Policy

```
podSelector:
  matchLabels:
    role: db
```



```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-policy
spec:
  podSelector:
    matchLabels:
      role: db
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          name: api-pod
    ports:
    - protocol: TCP
      port: 3306
```

kubectl create -f db-policy.yaml

Custom Resource Definitions (CRD)

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      type: front-end
  template:
    metadata:
      name: myapp-pod
      labels:
        type: front-end
    spec:
      containers:
        - image: nginx
```

```
kubectl create -f deployment.yml
# Output:
kubectl get deployments
# Output:
# NAME              DESIRED   CURRENT   UP-TO-DATE
AVAILABLE   AGE
kubectl delete -f deployment.yml
# Output:
# deployment "myapp-deployment" deleted
```

Custom Resource Definitions (CRD)

```yaml
apiVersion: flights.com/v1
kind: FlightTicket
metadata:
  name: my-flight-ticket
spec:
  from: Mumbai
  to: London
  number: 2
```

```
kubectl create -f flightticket.yml
kubectl get flightticket
kubectl delete -f flightticket.yml
```

Go

```go
package flightticket

import (
    // Imports omitidos
)

var controllerKind = apps.SchemeGroupVersion.WithKind("FlightTicket")

// Run inicia el loop del controller
func (dc *FlightTicketController) Run(workers int, stopCh <-chan struct{}) {
    // Lógica del loop del controller
}

// callBookFlightAPI se ejecuta cuando se crea un FlightTicket
func (dc *FlightTicketController) callBookFlightAPI(obj interface{}) {
    // Lógica para llamar a la API externa
}
```

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: flighttickets.flights.com
spec:
  group: flights.com
  scope: Namespaced
  names:
    plural: flighttickets
    singular: flightticket
    kind: FlightTicket
    shortNames:
      - ft
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                from:
                  type: string
                to:
                  type: string
                number:
                  type: integer
                  minimum: 1
```

kubectl create -f flightticket-custom-definition.yml
Output:
customresourcedefinition "flighttickets.flights.com" created

Custom Resource Definition (CRD)
Custom Controller
EtcdCluster
ETCD Controller
EtcdBackup
Backup Operator
EtcdRestore
Operator Framework

Operator Lifecycle Manager (OLM)

```
curl -sL https://github.com/operator-framework/operator-lifecycle-manager/releases/download/v0.19.1/install.sh | bash -s v0.19.1
```

```
kubectl create -f https://operatorhub.io/install/etcd.yaml
```

```
kubectl get csv -n my-etcd
```